https://leetcode.com/problems/binary-tree-preorder-traversal
https://leetcode.com/problems/binary-tree-inorder-traversal/description/
https://leetcode.com/problems/binary-tree-postorder-traversal
https://leetcode.com/problems/binary-tree-level-order-traversal

https://www.geeksforgeeks.org/problems/height-of-binary-tree/1
https://leetcode.com/problems/maximum-depth-of-binary-tree/description/

https://leetcode.com/problems/diameter-of-binary-tree/description/

https://leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal/description/
https://leetcode.com/problems/construct-binary-tree-from-inorder-and-postorder-traversal/description/

BUILDING A BINARY TREE

1. Node structure

Each binary-tree node stores a value and two child pointers. A missing child is represented by NULL/nullptr.

```
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) {
        data = val;
        left = NULL;
        right = NULL;
    }
};
```

2. Recursive construction using preorder input

A common beginner method reads values in preorder (Root, Left, Right). Use a special marker such as -1 to mean that a child position is empty. The marker is not a real node.

Example input: 1 2 4 -1 -1 5 -1 -1 3 -1 6 -1 -1

```
        1
      /   \\
     2     3
    / \\    \\
   4   5    6
```

```
Node* buildTree() {

    if (val == -1) return NULL;
```

```
    Node* root = new Node(val);
    root->left = buildTree();
    root->right = buildTree();

    return root;
}
```

Steps: (1) read a value; (2) if it is -1, return NULL; (3) create a node; (4) recursively build its left child; (5) recursively build its right child; (6) return the node. Input must follow the same preorder-with-null-markers format. If -1 can be a valid data value, use another marker.

```
int main() {
    // Input: 1 2 4 -1 -1 5 -1 -1 3 -1 6 -1 -1
    Node* root = buildTree();
    preorder(root);
}
```

Output: 1 2 4 5 3 6.

DIAMETER OF A BINARY TREE

Definition

The diameter is the length of the longest path between any two nodes. It does not have to pass through the root. Here, diameter is measured in edges.

For the sample tree, one longest path is 4 -> 2 -> 1 -> 3 -> 6, which contains 4 edges. Diameter = 4.

Three possibilities at each node

The longest path lies completely in the left subtree.

The longest path lies completely in the right subtree.

The longest path passes through the current node: left height + right height + 2 edges (with edge-based heights).

8. Simple approach — O(n²) worst case

```
int height(Node* root) {
    if (root == NULL) return -1;
    return 1 + max(height(root->left), height(root->right));
}

int diameter(Node* root) {
    if (root == NULL) return 0;

    int lh = height(root->left);
    int rh = height(root->right);
    int throughRoot = lh + rh + 2;

    int leftDia = diameter(root->left);
```

```
    int rightDia = diameter(root->right);

    return max(throughRoot, max(leftDia, rightDia));
}
```

Why O(n²) in the worst case? Height is recalculated at many nodes. In a skewed tree, each height call can scan a large remaining subtree, leading to repeated work.

9. Optimized approach — O(n)

Calculate height and update the best diameter in one DFS. Each call returns its subtree height; a reference variable stores the largest diameter seen.

```
int solve(Node* root, int& dia) {
    if (root == NULL) return -1;

    int lh = solve(root->left, dia);
    int rh = solve(root->right, dia);

    dia = max(dia, lh + rh + 2);
    return 1 + max(lh, rh);
}

int diameterOfTree(Node* root) {
    int dia = 0;
    solve(root, dia);
    return dia;
}
```

Node	Left height	Right height	Path through node
4	-1	-1	0
5	-1	-1	0
2	0	0	2
6	-1	-1	0
3	-1	0	1
1	1	1	4

The maximum path length is 4 edges. Optimized time: O(n), auxiliary recursion stack: O(h). Some platforms count diameter in nodes rather than edges; for a non-empty tree, node-count diameter is edge-count diameter + 1.

PATH SUM

https://leetcode.com/problems/path-sum/description/

11. Recursive solution

```
bool hasPathSum(Node* root, int targetSum) {
    if (root == NULL) return false;

    // A leaf must complete the sum
    if (root->left == NULL && root->right == NULL) {
        return root->data == targetSum;
    }

    int remaining = targetSum - root->data;

    return hasPathSum(root->left, remaining) ||
           hasPathSum(root->right, remaining);
}
```

Path Sum variations

Path Sum I: return true/false for at least one root-to-leaf path.

Path Sum II: return all root-to-leaf paths with the target sum; maintain a current path and backtrack.

Any-to-any path sum: path can start/end at any nodes; it is a different problem and requires a different approach.

Always read whether the problem asks for a root-to-leaf path, all paths, or any-to-any path.

BINARY SEARCH TREES (BST)

BST definition and property

A Binary Search Tree is a binary tree with an ordering rule. In the unique-key version used in these notes, every value in a node's left subtree is smaller than the node, and every value in its right subtree is greater. This rule applies to all descendants, not only immediate children.

Binary Tree vs BST

Binary Tree	Binary Search Tree
At most two children per node.	At most two children and ordered values.
No required value ordering.	Left subtree < node < right subtree (unique keys).
Inorder is not necessarily sorted.	Inorder gives ascending order.
Search can take O(n).	Search takes O(h).

Search in a BST

Compare the key with the current node. If equal, it is found. If smaller, go left; if greater, go right. This discards the other subtree.

```
bool searchBST(Node* root, int key) {
    if (root == NULL) return false;
    if (root->data == key) return true;

    if (key < root->data)
        return searchBST(root->left, key);

    return searchBST(root->right, key);
}
```

Iterative version:

```
bool searchBST(Node* root, int key) {
    Node* cur = root;
    while (cur != NULL) {
        if (cur->data == key) return true;
        if (key < cur->data) cur = cur->left;
        else cur = cur->right;
    }
    return false;
}
```

Example: search 7: compare with 8 (left), 3 (right), 6 (right), then reach 7.

Insert into a BST

Follow the same comparisons as search. When the required child pointer is NULL, create and attach the new node. This version ignores duplicates.

```
Node* insertBST(Node* root, int key) {
    if (root == NULL) return new Node(key);

    if (key < root->data)
        root->left = insertBST(root->left, key);
    else if (key > root->data)
        root->right = insertBST(root->right, key);

    return root;
}
```

Example: insert 5: 5 < 8, go left; 5 > 3, go right; 5 < 6, go left; attach 5 as left child of 6. Assign the returned pointer when connecting subtrees.

17. Minimum and maximum

The minimum is the leftmost node. The maximum is the rightmost node.

```
Node* findMin(Node* root) {
    if (root == NULL) return NULL;
```

```
    while (root->left != NULL) root = root->left;
    return root;
}

Node* findMax(Node* root) {
    if (root == NULL) return NULL;
    while (root->right != NULL) root = root->right;
    return root;
}
```

Delete from a BST

Deletion has three cases. We must preserve the BST ordering after removing the key.

Case 1 — Leaf node

Delete the node and return NULL to its parent.

Case 2 — One child

Replace the node with its only child. Return that child so the parent reconnects to it.

Case 3 — Two children

Find the inorder successor (smallest node in the right subtree) or inorder predecessor (largest node in the left subtree). Copy its value into the node, then delete that successor/predecessor from its old location.

18.1 C++ deletion code

```
Node* deleteBST(Node* root, int key) {
    if (root == NULL) return NULL;

    if (key < root->data) {
        root->left = deleteBST(root->left, key);
    }
    else if (key > root->data) {
        root->right = deleteBST(root->right, key);
    }
    else {
        // No child
        if (root->left == NULL && root->right == NULL) {
            delete root;
            return NULL;
        }

        // Only right child
        if (root->left == NULL) {
            Node* child = root->right;
            delete root;
            return child;
        }

        // Only left child
        if (root->right == NULL) {
            Node* child = root->left;
            delete root;
            return child;
        }
```

```
        // Two children: use inorder successor
        Node* succ = findMin(root->right);
        root->data = succ->data;
        root->right = deleteBST(root->right, succ->data);
    }
    return root;
}
```

Use it as: root = deleteBST(root, key). This code uses findMin() from the previous section and assumes duplicate keys are not stored.

Inorder traversal of a BST

Inorder is Left -> Root -> Right. Because every left-subtree value is smaller and every right-subtree value is greater, inorder traversal of a BST prints values in ascending order.

```
void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
```

BST complexity

Operation	In terms of height h	Balanced BST	Skewed BST
Search	O(h)	O(log n)	O(n)
Insert	O(h)	O(log n)	O(n)
Delete	O(h)	O(log n)	O(n)
Find min/max	O(h)	O(log n)	O(n)
Inorder traversal	O(n)	O(n)	O(n)

A BST is not automatically balanced. Inserting sorted values into a basic BST can create a skewed tree, making it behave like a linked list.